

(An Autonomous Institution, Affiliated to Anna University, Chennai)

AI MOOD PREDICTOR – LSTM EMOTION CLASSIFICATION

Submitted by

AJAY ANAND (241501012)
AKSHAYA K (241501016)
AYSHWARYA (241501028)

AI23531 DEEP LEARNING

Department of Artificial Intelligence and Machine Learning

Rajalakshmi Engineering College,
Rajalakshmi Nagar, Thandalam

Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

NAME	UNIVERSITY REGISTER NO.
AJAY ANAND	**241501012**
AKSHAYA K	**241501016**
AYSHWARYA	**241501028**

ACADEMIC YEAR 2026 – 2027 **SEMESTER** V

BRANCH Artificial Intelligence and Machine Learning

Certified that this is the bonafide record of work done by the above students in the Mini Project titled **AI Mood Predictor – LSTM Emotion Classification** in the subject **AI23531 Deep Learning** during the year **2026 - 2027**.

Signature of Faculty – in – Charge

Submitted for the Practical Examination held on

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ABSTRACT

Identifying the mood expressed in a piece of text is a common task in natural language processing, with uses in feedback analysis and conversational systems. AI Mood Predictor is a deep learning-based system that classifies a short English sentence into one of four mood categories: Happy, Sad, Angry or Neutral. The system uses a sequential neural network developed with TensorFlow and Keras, consisting of an Embedding layer with 64 dimensions, an LSTM layer with 64 units, two Dropout layers, a Dense layer with 32 ReLU units and a four-unit softmax output layer.

A labelled dataset of 60 sentences, 15 for each mood, is encoded with the scikit-learn LabelEncoder, converted to integer sequences using the Keras Tokenizer with an out-of-vocabulary token, padded to a length of 20 and divided by stratified sampling into 38 training, 10 validation and 12 test sentences. The model is trained for 30 epochs with the Adam optimizer and a batch size of 4. A console-based prediction function displays the predicted mood, a matching emoji, the confidence and the probability of every class, and a separate evaluation script computes accuracy, precision, recall, F1-score and the confusion matrix.

On the 12-sentence test set the model obtained an accuracy of 25.00% with a test loss of 1.8728. All 12 test sentences were predicted as Sad, which gives a macro F1-score of 0.10. These results indicate that the very small dataset is a likely reason why the LSTM could not learn to separate the four classes reliably, and they identify a larger dataset as the most important direction for future improvement.

Keywords: Deep Learning, LSTM, Recurrent Neural Network, Emotion Classification, Text Classification, Word Embedding, TensorFlow, Keras, Natural Language Processing.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	III
1	**INTRODUCTION**	1
2	**LITERATURE REVIEW**	3
3	**SYSTEM REQUIREMENTS**	5
	3.1 HARDWARE REQUIREMENTS	5
	3.2 SOFTWARE REQUIREMENTS	5
	3.3 LIBRARIES USED	5
4	**SYSTEM OVERVIEW**	6
	4.1 EXISTING SYSTEM	6
	4.1.1 DRAWBACKS OF EXISTING SYSTEM	6
	4.2 PROPOSED SYSTEM	6
	4.2.1 ADVANTAGES OF PROPOSED SYSTEM	6
5	**SYSTEM IMPLEMENTATION**	7
	5.1 SYSTEM ARCHITECTURE DIAGRAM	7
	5.2 SYSTEM FLOW	8
	5.3 LIST OF MODULES	9
	5.4 MODULE DESCRIPTION	10
6	**RESULTS AND DISCUSSION**	13
7	**APPENDIX**	18
	7.1 SAMPLE CODE	18
	7.2 OUTPUT SCREENSHOTS	22
	REFERENCES	27

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

Text is one of the most common ways in which people express how they feel. Messages, reviews, posts and short comments all carry a mood, and being able to identify that mood automatically is a well-known problem in natural language processing. Mood or emotion classification can support applications such as the analysis of feedback and the development of conversational systems that respond to the feeling behind a sentence.

Early approaches to this problem relied on manually prepared word lists or on features designed by hand. Deep learning offers an alternative in which the model learns useful representations directly from labelled examples. Recurrent neural networks are designed for sequential data, and the Long Short-Term Memory (LSTM) network [1] is a recurrent architecture that uses gated memory cells to retain information while a sentence is processed word by word.

This project, **AI Mood Predictor**, applies an Embedding and LSTM network to classify a short English sentence into one of four moods: Happy, Sad, Angry or Neutral. The dataset used in this project is sourced from our own dataset , while the model design, training pipeline, evaluation and prediction application were developed by the project team using TensorFlow and Keras.

In the implementation, a working dataset of 60 labelled sentences, 15 for each mood, is converted into integer sequences, padded to a fixed length and used to train the network. After training, a console-based function accepts a sentence typed by the user and displays the predicted mood, an emoji, the confidence and the probability of each of the four classes. A separate evaluation script reports accuracy, precision, recall, F1-score and the confusion matrix on a held-out test set.

Because the dataset is very small, the project also examines how far an LSTM can be trained on so few examples. The results are reported as obtained, including their limitations, in Chapter 6.

1.2 OBJECTIVES

The main objectives of the project are:

- To develop a deep learning model (Embedding + LSTM) that predicts the mood of an input sentence as Happy, Sad, Angry or Neutral.
- To prepare a labelled dataset of short sentences and encode the mood labels as integers.

- To convert sentences into integer sequences using the Keras Tokenizer with an out-of-vocabulary token and pad them to a length of 20.
- To divide the data into stratified training, validation and test sets.
- To train the model for 30 epochs and monitor the training and validation accuracy and loss.
- To evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- To provide a console-based prediction function that displays the predicted mood, emoji, confidence and class probabilities.
- To study the limitations of training an LSTM on a very small dataset.

1.3 SCOPE OF THE PROJECT

The project focuses on sentence-level mood classification of short English sentences into four categories: Happy, Sad, Angry and Neutral. The model is built, trained, evaluated and used for prediction in a Google Colab notebook, and user input is taken through a console prompt.

The project is an academic prototype that demonstrates a complete deep learning workflow for text classification. It does not provide a verified detection of emotion in real-world text. Its predictions are limited to the four categories it was trained on and depend on the small training dataset. The system is not intended for clinical, safety-related or other decision-making use.

CHAPTER 2
LITERATURE REVIEW

2.1 DEEP LEARNING FOR TEXT CLASSIFICATION

Text classification assigns a category to a piece of text. Conventional approaches represent a document by the words it contains, for example as a bag of words, and pass these features to a separate classifier. Deep learning models instead accept the text as a sequence of numerical tokens and learn their own representations during training.

To be used by a neural network, text must first be converted into numbers. A common method is to build a vocabulary, replace each word by its integer index and pad all sequences to the same length. This is the approach followed in this project.

2.2 RECURRENT NEURAL NETWORKS AND LSTM

A recurrent neural network processes a sequence one element at a time and keeps a hidden state that summarizes what it has read so far. Simple recurrent networks have difficulty learning dependencies that span many steps. Hochreiter and Schmidhuber [1] introduced the Long Short-Term Memory network to address this problem by using a memory cell controlled by gates.

LSTM layers are available in Keras [7] and are widely used for sequence data such as text. In this project, an LSTM layer with 64 units reads the sequence of word vectors for a sentence and produces a single 64-dimensional summary vector, which is then passed to the classification layers.

2.3 WORD EMBEDDINGS

A word embedding represents each word as a dense vector of real numbers, so that words can be compared and combined numerically. Mikolov et al. [2] showed that such vectors can be learned efficiently from large text collections.

In this project, the Embedding layer is not pre-trained. Its 64-dimensional vectors are initialized at random and learned together with the rest of the network from the project's own 60 sentences.

2.4 REGULARIZATION AND OPTIMIZATION

Neural networks trained on small datasets can fit the training examples without generalizing well. Dropout [4] reduces this risk by randomly switching off a fraction of units during training. The model in this project uses Dropout layers with rates of 0.3 and 0.2.

The network weights are updated using the Adam optimizer [3], an adaptive method that adjusts the learning rate for each parameter. The loss function is sparse categorical cross-entropy, which suits integer class labels.

2.5 EMOTION DATASET

The dataset used in this project is sourced from our own dataset. In the implementation, the working dataset contains 60 short sentences defined in the source code, with 15 sentences for each of the four moods: Happy, Sad, Angry and Neutral. The complete list of sentences is given in Table 7.2.

A dataset of this size is useful for demonstrating the workflow, but results obtained on it should not be assumed to hold for larger collections or for text from other sources.

2.6 RESEARCH GAP

AI Mood Predictor combines a small Embedding and LSTM network with a console prediction function and an evaluation script. The project does not propose new architecture. Its contribution is a complete, reproducible workflow for four-class mood classification, together with an evaluation that goes beyond accuracy by reporting precision, recall, F1-score and the confusion matrix. This evaluation shows clearly how a very small dataset affects the behavior of the model.

CHAPTER 3
SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

- Computer: any laptop or desktop with a modern web browser and an internet connection for Google Collab.
- Runtime: Google Collab hosted runtime. The exact processor and memory allocation were not recorded [Add CPU/RAM detail if required]
- GPU: not required, because the model and dataset are small; training steps took about 15–20 ms in the recorded run.

3.2 SOFTWARE REQUIREMENTS

- Development Environment: Google Colab notebook (Linux-based cloud runtime).
- Programming Language: Python 3.13.
- Deep Learning Framework: TensorFlow 2.21.0.
- Model API: Keras (Sequential model, Embedding, LSTM, Dense and Dropout layers).
- Machine Learning Utilities: scikit-learn.
- Numerical and Data Processing: NumPy and pandas.
- Visualization: Matplotlib.
- Dataset: emotion sentences labelled Happy, Sad, Angry and Neutral (Kaggle source).

3.3 LIBRARIES USED

Library	Purpose
TensorFlow / Keras	Building, training and running the Embedding–LSTM model (Sequential, Embedding, LSTM, Dense, Dropout)
Keras Tokenizer and pad_sequences	Converting sentences into integer sequences and padding them to a fixed length
scikit-learn	LabelEncoder, train_test_split, accuracy, precision, recall, F1-score, classification report and confusion matrix
NumPy	Array preparation (int32 arrays) and argmax for selecting the predicted class
pandas	Class-wise metrics table and CSV export of predictions and metrics
Matplotlib	Training and validation curves, confusion matrix and metrics bar chart
google.colab (files)	Downloading the generated evaluation report from Colab

Table 3.1: Libraries Used

CHAPTER 4
SYSTEM OVERVIEW

4.1 EXISTING SYSTEM

Mood in text is traditionally judged by a human reader, or identified automatically by matching words against manually prepared lists of emotion-related keywords. Another established approach represents each sentence as a bag of words and passes it to a conventional classifier. These methods are simple to apply, but they treat words largely independently of one another and depend on features or word lists prepared in advance.

4.1.1 DRAWBACKS OF EXISTING SYSTEM

- Manual reading of large amounts of text is slow and can vary between readers.
- Keyword matching cannot handle words that are not in the prepared list.
- Bag-of-words representations ignore the order of words in a sentence.
- Hand-designed features require effort and do not learn word representations from the training data.

4.2 PROPOSED SYSTEM

AI Mood Predictor is a deep learning-based system that classifies a short sentence as Happy, Sad, Angry or Neutral using an Embedding and LSTM network.

Sentences are converted into integer sequences using the Keras Tokenizer, with an out-of-vocabulary token for unseen words, and padded to a length of 20. The Embedding layer converts each token into a 64-dimensional vector, the LSTM layer reads the sequence, and a softmax layer produces a probability for each of the four moods.

The `predict_emotion()` function applies the same preprocessing to a new sentence, selects the class with the highest probability using argmax, and prints the predicted mood, an emoji, the confidence and the probability of every class.

4.2.1 ADVANTAGES OF PROPOSED SYSTEM

- Reads each sentence as an ordered sequence of words.
- Learns word representations through the Embedding layer and handles unseen words with an out-of-vocabulary token.
- Gives a probability for every mood, not only the final label.

- Provides an interactive console prompt for testing new sentences.
- Includes an evaluation script reporting accuracy, precision, recall, F1-score and a confusion matrix.

CHAPTER 5
SYSTEM IMPLEMENTATION

5.1 SYSTEM ARCHITECTURE DIAGRAM

The AI Mood Predictor system consists of a training pipeline and a prediction pipeline. In the training pipeline, the labelled dataset is encoded, tokenized, padded and split into training, validation and test sets. The Embedding–LSTM model is then built, compiled, trained and evaluated.

In the prediction pipeline, the user types a sentence at the console prompt. The sentence is tokenized and padded using the same tokenizer that was fitted on the dataset, and the trained model produces four probabilities. The class with the highest probability is selected and the mood, emoji, confidence and class probabilities are displayed.

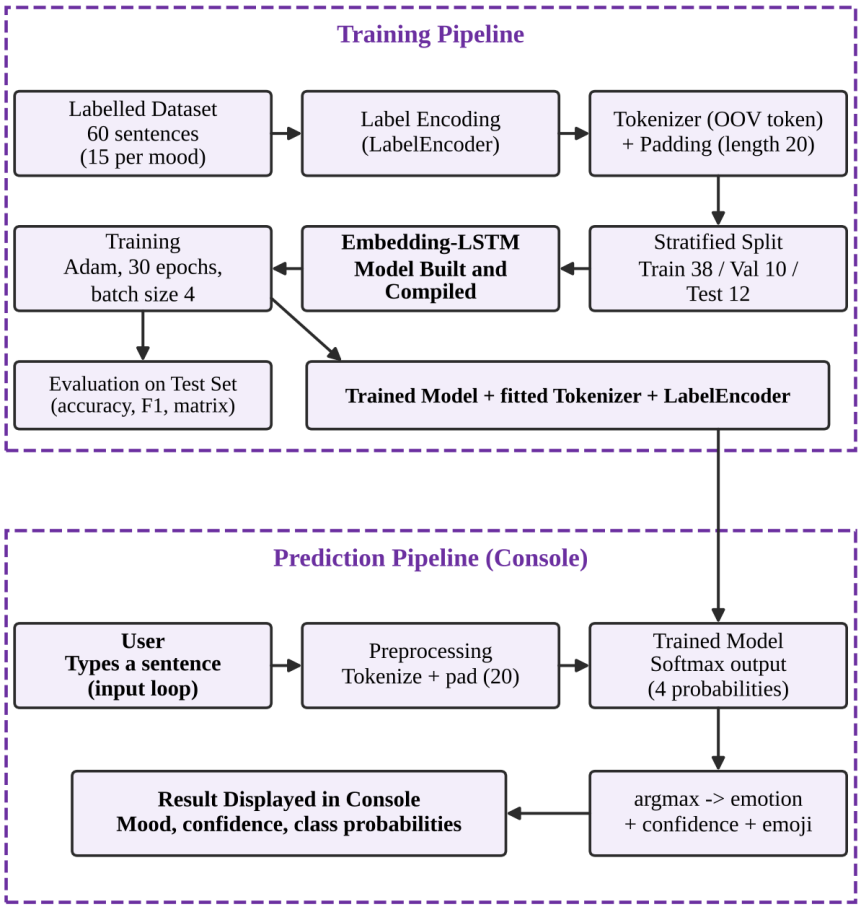


Fig. 5.1: System Architecture

5.2 SYSTEM FLOW

The system follows the sequence of operations given below.

1. A dataset of 60 labelled sentences is created, with 15 sentences for each mood.
2. The mood labels are encoded as integers using LabelEncoder.
3. The sentences are converted into integer sequences and padded to a length of 20.
4. The data is split into training, validation and test sets using stratified sampling.
5. The Embedding–LSTM model is built.
6. The model is compiled with the Adam optimizer and sparse categorical cross-entropy loss.
7. The model is trained for 30 epochs with a batch size of 4.
8. The model is evaluated on the test set and the training and validation curves are plotted.
9. A sentence entered by the user is preprocessed and passed to the model.
10. The predicted mood, emoji, confidence and class probabilities are displayed.

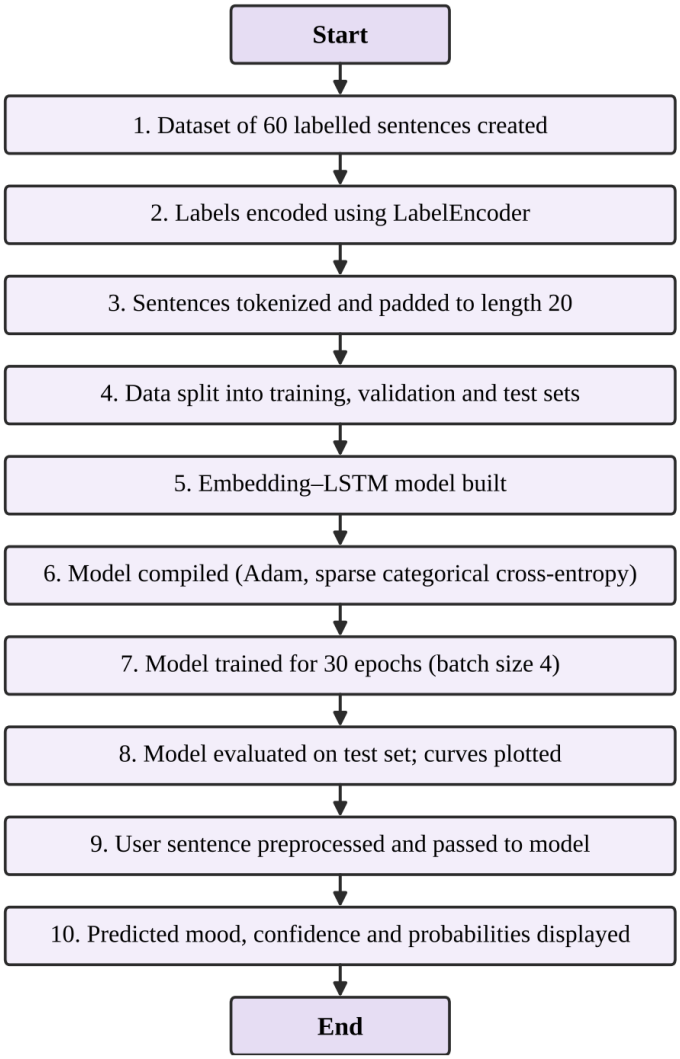


Fig. 5.2: System Flow Diagram

5.3 LIST OF MODULES

The application consists of the following modules:

- Dataset Creation and Label Encoding Module
- Text Preprocessing Module
- Data Splitting Module
- Model Building Module
- Model Training Module
- Model Evaluation Module
- Emotion Prediction Module

5.4 MODULE DESCRIPTION

5.4.1 Dataset Creation and Label Encoding Module

This module defines the list of 60 short sentences and the matching list of mood labels. There are 15 sentences for each of the four moods: Happy, Sad, Angry and Neutral. The complete list is given in Table 7.2.

The text labels are converted into integers using the scikit-learn `LabelEncoder`. The encoder orders the classes alphabetically, so Angry is encoded as 0, Happy as 1, Neutral as 2 and Sad as 3. The same order is used to name the four output probabilities.

5.4.2 Text Preprocessing Module

This module prepares the sentences for the neural network. A Keras `Tokenizer` is created with `num_words=1000` and `oov_token="<OOV>"` and is fitted on the sentences. Each sentence is then converted into a sequence of integers and padded with zeros at the end (`padding="post"`) to a fixed length of 20. The resulting arrays are stored as 32-bit integers.

For this dataset the tokenizer finds 116 distinct words. Together with the out-of-vocabulary token and the padding index, this gives a vocabulary size of 118 for the Embedding layer. The longest sentence contains 6 words, so every sequence contains at least 14 padding values.

5.4.3 Data Splitting Module

The data is divided in two steps using `train_test_split` with `random_state=42` and stratified sampling. First, 20% of the data is kept aside as the test set. Then 20% of the remaining sentences is taken as the validation set. Table 5.1 shows the resulting sizes.

Subset	Sentences	Purpose
Training set	38	Used to update the model weights
Validation set	10	Used to monitor the model during training
Test set	12 (3 per mood)	Used for the final evaluation
Total	60	15 sentences per mood

Table 5.1: Data Split

5.4.4 Model Building Module

The model is a Keras `Sequential` network with the layers listed in Table 5.2 and shown in Fig. 5.3. The Embedding layer converts each token into a 64-dimensional vector. The LSTM layer with

64 units reads the sequence of vectors and returns a single vector for the sentence. Two Dropout layers with rates of 0.3 and 0.2 reduce overfitting. A Dense layer with 32 ReLU units is followed by a Dense output layer with 4 units and softmax activation, which gives one probability for each mood.

Layer	Configuration	Output Shape
Embedding	input_dim = 118 (vocabulary size), output_dim = 64	(None, 20, 64)
LSTM	64 units	(None, 64)
Dropout	rate = 0.3	(None, 64)
Dense	32 units, ReLU activation	(None, 32)
Dropout	rate = 0.2	(None, 32)
Dense	4 units, softmax activation	(None, 4)

Table 5.2: Model Architecture

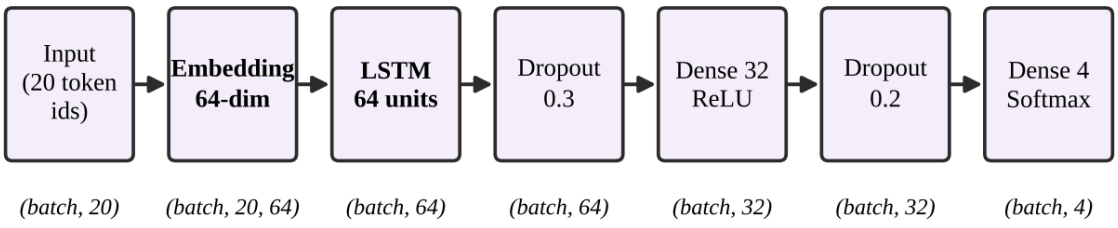


Fig. 5.3: Model Architecture

5.4.5 Model Training Module

The model is compiled and trained with the settings in Table 5.3. With 38 training sentences and a batch size of 4, each epoch consists of 10 steps. The validation set is evaluated at the end of every epoch, and the accuracy and loss of both sets are stored in the training history.

Parameter	Value
Optimizer	Adam
Loss function	Sparse categorical cross-entropy
Metric	Accuracy
Epochs	30
Batch size	4
Steps per epoch	10
Validation data	10 sentences

Table 5.3: Training Configuration

5.4.6 Model Evaluation Module

After training, the model is evaluated on the test set to obtain the test loss and accuracy, and the training and validation accuracy and loss are plotted against the epoch number.

A separate evaluation script then predicts the test sentences and computes the accuracy, macro precision, macro recall, macro F1-score, weighted F1-score and test loss. It also prints a classification report, plots the confusion matrix and a bar chart of precision, recall and F1-score for each mood, and saves the predictions and metrics as CSV files (`test_predictions.csv`, `classwise_metrics.csv`, `overall_metrics.csv`) together with a written report (`evaluation_report.txt`).

5.4.7 Emotion Prediction Module

The `predict_emotion(sentence)` function converts the input sentence into a padded sequence using the fitted tokenizer, obtains the four class probabilities from the model, and selects the class with the highest probability using argmax. The class index is converted back into the mood name using the label encoder.

The function prints the input sentence, the predicted mood, an emoji for that mood, the confidence as a percentage and the probability of each class. The emoji used are one for each of Happy, Sad, Angry and Neutral. The confidence is the highest softmax output of the model and should not be interpreted as a guarantee that the prediction is correct.

The program first tests four fixed sentences and then enters a loop that asks the user to type a sentence. Typing exit ends the program, and empty input is ignored.

CHAPTER 6
RESULTS AND DISCUSSION

6.1 IMPLEMENTATION RESULTS

The AI Mood Predictor was executed successfully in Google Colab using Python 3.13 and TensorFlow 2.21.0. The dataset was prepared, the model was trained for 30 epochs, the test set was evaluated, and the prediction function was used on both fixed and user-typed sentences. Table 6.1 summarises the setup and the main results of the recorded run.

Item	Observed Value
Framework	TensorFlow 2.21.0 (Keras)
Training / validation / test sentences	38 / 10 / 12
Epochs and batch size	30 and 4
Final training accuracy and loss (epoch 30)	0.5789 and 1.0686
Final validation accuracy and loss (epoch 30)	0.4000 and 1.2355
Highest validation accuracy	0.5000 (epoch 26)
Test accuracy	25.00 %
Test loss	1.8728

Table 6.1: Experimental Setup and Main Results

The training progress at selected epochs is shown in Table 6.2. The complete training log is given in Fig. 7.2 and Fig. 7.3, and the training and validation curves are shown in Fig. 7.4.

Epoch	Training Accuracy	Training Loss	Validation Accuracy	Validation Loss
1	0.1579	1.3908	0.2000	1.3891
5	0.3421	1.3884	0.2000	1.3874
10	0.2368	1.3917	0.3000	1.3855
15	0.4737	1.2245	0.4000	1.2317
20	0.5000	0.9894	0.3000	1.5537
25	0.4474	1.0546	0.3000	1.5489
30	0.5789	1.0686	0.4000	1.2355

Table 6.2: Training Progress at Selected Epochs

Table 6.3 lists the predictions made for the four fixed test sentences and for one sentence typed by the user. The corresponding console output is shown in Fig. 7.5 and Fig. 7.6.

Input Sentence	Predicted Mood	Angry	Happy	Neutral	Sad
I got selected for my dream job	Sad (38.39 %)	1.48	33.00	27.13	38.39
I failed my final examination	Sad (38.30 %)	1.50	32.95	27.26	38.30
This situation makes me extremely angry	Angry (94.35 %)	94.35	0.12	5.36	0.17
I went to the library today	Sad (37.90 %)	1.61	32.79	27.70	37.90
I am happy (typed by user)	Sad (38.38 %)	1.47	32.98	27.16	38.38

Table 6.3: Sample Predictions (class probabilities in %)

6.2 RESULTS DISCUSSION

The complete workflow of dataset preparation, tokenization, training, evaluation and interactive prediction ran without errors. The model summary in Fig. 7.1 shows question marks for the output shapes and zero parameters because it was printed before the model had been built. A warning in the same output notes that the `input_length` argument of the Embedding layer is deprecated. Neither of these affected the training of the model.

During the first 12 epochs, both the training loss and the validation loss stayed close to 1.39. This is the loss of a model that assigns the same probability to all four classes (the natural logarithm of 4 is about 1.386), so the network had not yet learned anything useful about the sentences. From epoch 13 onwards the training loss began to fall, reaching 1.2245 at epoch 15 and 1.0686 at epoch 30, with its lowest value of 0.8884 at epoch 28.

The validation results were much less stable. The validation loss varied between about 1.17 and 1.97 after epoch 14, and the validation accuracy peaked at 0.5000 at epoch 26 before ending at 0.4000. With only 10 validation sentences, each sentence changes the accuracy by 10 percentage points, so the validation curves in Fig. 7.4 are step-like and noisy. The training accuracy reached 0.5789, which is higher than the validation accuracy, but both are low.

On the test set the model obtained an accuracy of 25.00% and a loss of 1.8728. The accuracy is the same as would be expected from choosing one of four equally likely classes at random. The confusion matrix in Table 6.7 shows that all 12 test sentences were predicted as Sad. The recall

for Sad is therefore 1.0 and its precision is 0.25, while the precision, recall and F1-score of Angry, Happy and Neutral are all 0. The macro and weighted F1-scores are 0.10.

The sample predictions in Table 6.3 show the same behaviour. For four of the five sentences, the output probabilities were almost identical (Sad about 38%, Happy about 33%, Neutral about 27% and Angry about 1.5%), so the output hardly changed with the input sentence. The confidence values of these predictions are below 40%. Sentences such as “I got selected for my dream job” and “I am happy” were predicted as Sad, which is not the expected mood. The only distinct output was for “This situation makes me extremely angry”, which was predicted as Angry with a confidence of 94.35%. A likely reason is that several of its words, such as angry, extremely, this and makes me, also occur in sentences labelled Angry in the dataset.

These results should not be taken to mean that LSTM networks are unsuitable for mood classification. They reflect the conditions of this experiment: only 38 training sentences, 116 distinct words so that most words appear in only a few sentences, an Embedding layer learned from scratch, and a small batch size of 4 that makes the updates noisy. These factors are likely to have contributed to the result, but their individual effects were not tested in this project.

6.3 PERFORMANCE EVALUATION

The following metrics were used to evaluate the classification model.

Metric	Description
Accuracy	Proportion of test sentences that are classified correctly.
Precision	Proportion of sentences predicted as a given mood that truly belong to that mood.
Recall	Proportion of sentences of a given mood that are correctly identified.
F1-Score	Harmonic mean of precision and recall.
Confusion Matrix	Shows correct and incorrect classifications for each pair of actual and predicted moods.

Table 6.4: Evaluation Metrics

The values obtained on the 12-sentence test set are given in Table 6.5 and Table 6.6. Macro averages give each mood equal weight. The evaluation output is shown in Fig. 7.7 and Fig. 7.9, and the bar chart of class-wise metrics in Fig. 7.10.

Metric	Value
Accuracy	0.2500 (25.00 %)
Macro Precision	0.0625
Macro Recall	0.2500
Macro F1-Score	0.1000
Weighted F1-Score	0.1000
Test Loss	1.8728

Table 6.5: Overall Test Performance

Mood	Precision	Recall	F1-Score	Support
Angry	0.00	0.00	0.00	3
Happy	0.00	0.00	0.00	3
Neutral	0.00	0.00	0.00	3
Sad	0.25	1.00	0.40	3

Table 6.6: Class-wise Test Performance

Actual \ Predicted	Angry	Happy	Neutral	Sad
Angry	0	0	0	3
Happy	0	0	0	3
Neutral	0	0	0	3
Sad	0	0	0	3

Table 6.7: Confusion Matrix on the Test Set

6.4 LIMITATIONS

The current implementation has several limitations.

- The training set contains only 38 sentences and the test set only 12, so the reported metrics are based on very few examples and can change considerably with a different split.
- The model did not learn to separate Happy, Neutral and Sad on the test set, and all test sentences were predicted as Sad.
- The tokenizer was fitted on all 60 sentences before the data was split. For a stricter evaluation it should be fitted on the training sentences only.

- The results come from a single training run and a single data split. Training is random, so repeated runs may give different values.
- The Embedding layer is learned from scratch and no pre-trained word vectors are used.
- The confidence value is the highest softmax output of the model. It is not a calibrated probability and does not guarantee that a prediction is correct.
- The model is limited to four moods and to short English sentences similar to those in the dataset.
- The application runs as a console program and has no graphical or web interface.

6.5 CONCLUSION

AI Mood Predictor implements a complete deep learning workflow for four-class mood classification of short sentences using an Embedding and LSTM network built with TensorFlow and Keras. The workflow covers dataset preparation, tokenization and padding, stratified data splitting, training, evaluation with accuracy, precision, recall, F1-score and a confusion matrix, and an interactive console function that reports the predicted mood, emoji, confidence and class probabilities.

On the 12-sentence test set the model obtained an accuracy of 25.00% and predicted every sentence as Sad. The evaluation therefore shows that the model trained on 60 sentences did not generalize, and it also shows why accuracy alone should be supported by per-class metrics and a confusion matrix. The project nevertheless provides a working and reproducible baseline.

Future improvements include:

- Training on a much larger dataset, such as the complete Kaggle emotion dataset.
- Fitting the tokenizer on the training sentences only.
- Using pre-trained word embeddings and comparing alternative architectures such as GRU or bidirectional LSTM.
- Tuning hyperparameters such as batch size, number of units and number of epochs, and using early stopping.
- Repeating the experiment with several random splits or cross-validation.
- Adding a graphical or web interface for entering sentences.

CHAPTER 7
APPENDIX

PROJECT FILES

The files produced by the evaluation script are listed in Table 7.1.

File Name	Description
Mini project notebook	Google Colab notebook containing the training, prediction and evaluation code
confusion_matrix.png	Confusion matrix of the test set (300 dpi)
emotion_metrics.png	Bar chart of precision, recall and F1-score for each mood
test_predictions.csv	Actual emotion, predicted emotion, correctness and confidence for each test sentence
classwise_metrics.csv	Precision, recall and F1-score for each mood
overall_metrics.csv	Accuracy, macro precision, macro recall, macro F1-score, weighted F1-score and test loss
evaluation_report.txt	Written evaluation report with the overall results, classification report, findings and limitation

Table 7.1: Project Files

7.1 SAMPLE CODE

The 60 labelled sentences used in the project are listed in Table 7.2. In the code, they form the list `texts`.

Happy	Sad	Angry	Neutral
I am very happy today	I am feeling very sad	I am very angry	I went to college today
I feel amazing and wonderful	Today was a terrible day	This makes me furious	The weather is normal
Today is the best day	I feel lonely	I hate this situation	I had lunch with my friend
I am so excited	I am disappointed	I am extremely annoyed	I am going to class
I got good news today	I failed my exam	This is so irritating	Today is Monday
I am feeling great	I feel like crying	I am frustrated	I bought a new book
I am really joyful	I am unhappy today	Stop bothering me	The bus arrived on time
I love this day	Nothing is going well	I am angry about this	I completed my homework
I am extremely happy	I lost something important	This makes me mad	I watched a movie
Everything is going perfectly	I miss my old friends	I cannot tolerate this	I am sitting in my room

Happy	Sad	Angry	Neutral
I won the competition	I feel heartbroken	I am tired of this nonsense	I went to the library
I received a wonderful gift	I received bad news	That behavior made me angry	I attended my class
My friends made me happy	My result was disappointing	I am really irritated	I had breakfast
I am enjoying my day	I feel terrible	This is unacceptable	I travelled by bus
I feel fantastic	I am having a bad day	I am losing my patience	I opened my laptop

Table 7.2: Labelled Sentences Used in the Project

Data preparation:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# texts = [ ... 60 sentences, listed in Table 7.2 ... ]
labels = (
    ["Happy"] * 15 + ["Sad"] * 15 +
    ["Angry"] * 15 + ["Neutral"] * 15
)

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(labels)

tokenizer = Tokenizer(num_words=1000, oov_token="<OOV>")
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)

X = pad_sequences(sequences, maxlen=20, padding="post")
X = np.array(X, dtype=np.int32)
y = np.array(y, dtype=np.int32)
```

Data splitting, model building, training and testing:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.20,
    random_state=42, stratify=y_train
)

vocab_size = min(1000, len(tokenizer.word_index) + 1)
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=64, input_length=20),
    LSTM(64),
    Dropout(0.3),
    Dense(32, activation="relu"),
    Dropout(0.2),
    Dense(4, activation="softmax")
```

```
])
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
model.summary()

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=30, batch_size=4, verbose=1
)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy:", round(accuracy * 100, 2), "%")
print("Test Loss:", round(loss, 4))
```

Prediction function and console loop:

```
def predict_emotion(sentence):
    sequence = tokenizer.texts_to_sequences([sentence])
    padded = pad_sequences(sequence, maxlen=20, padding="post")
    prediction = model.predict(padded, verbose=0)[0]
    index = np.argmax(prediction)
    emotion = label_encoder.inverse_transform([index])[0]
    confidence = prediction[index] * 100
    emoji = {"Happy": "😊", "Sad": "😢",
             "Angry": "😡", "Neutral": "😐"}
    print("Input:", sentence)
    print("Predicted Emotion:", emotion)
    print("Emoji:", emoji[emotion])
    print("Confidence:", round(confidence, 2), "%")
    print("\nProbabilities:")
    for name, probability in zip(label_encoder.classes_, prediction):
        print(name + ":", round(probability * 100, 2), "%")

predict_emotion("I got selected for my dream job")
predict_emotion("I failed my final examination")
predict_emotion("This situation makes me extremely angry")
predict_emotion("I went to the library today")

while True:
    sentence = input("\nEnter a sentence (type exit to stop): ")
    if sentence.lower().strip() == "exit":
        print("Program ended.")
        break
    if sentence.strip():
        predict_emotion(sentence)
```

Performance evaluation (summary of the evaluation script):

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
    f1_score, classification_report, confusion_matrix, ConfusionMatrixDisplay)

y_prob = model.predict(X_test, verbose=0)
y_pred = np.argmax(y_prob, axis=1)
classes = label_encoder.classes_

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average="macro", zero_division=0)
recall = recall_score(y_test, y_pred, average="macro", zero_division=0)
f1 = f1_score(y_test, y_pred, average="macro", zero_division=0)
weighted_f1 = f1_score(y_test, y_pred, average="weighted", zero_division=0)
```

```
loss, _ = model.evaluate(X_test, y_test, verbose=0)

print(classification_report(y_test, y_pred, labels=np.arange(len(classes)),
      target_names=classes, zero_division=0, digits=4))

cm = confusion_matrix(y_test, y_pred, labels=np.arange(len(classes)))
ConfusionMatrixDisplay(confusion_matrix=cm,
                       display_labels=classes).plot(cmap="Blues")
plt.savefig("confusion_matrix.png", dpi=300)

# The script also plots precision/recall/F1 for each mood and saves
# test_predictions.csv, classwise_metrics.csv, overall_metrics.csv
# and evaluation_report.txt.
```

7.2 OUTPUT SCREENSHOTS

```
TensorFlow: 2.21.0
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)
dropout_1 (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)
```

Fig. 7.1: Library versions, warning and model summary

```
Training model...

Epoch 1/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 4s 54ms/step - accuracy: 0.1579 - loss: 1.3908 - val_accuracy: 0.2000 - val_loss: 1.3891
Epoch 2/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.2368 - loss: 1.3958 - val_accuracy: 0.2000 - val_loss: 1.3914
Epoch 3/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.2632 - loss: 1.3860 - val_accuracy: 0.2000 - val_loss: 1.3878
Epoch 4/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.2105 - loss: 1.3917 - val_accuracy: 0.2000 - val_loss: 1.3867
Epoch 5/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.3421 - loss: 1.3884 - val_accuracy: 0.2000 - val_loss: 1.3874
Epoch 6/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.2895 - loss: 1.3847 - val_accuracy: 0.2000 - val_loss: 1.3864
Epoch 7/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1579 - loss: 1.3870 - val_accuracy: 0.3000 - val_loss: 1.3847
Epoch 8/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.2895 - loss: 1.3805 - val_accuracy: 0.3000 - val_loss: 1.3850
Epoch 9/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.3421 - loss: 1.3870 - val_accuracy: 0.2000 - val_loss: 1.3866
Epoch 10/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.2368 - loss: 1.3917 - val_accuracy: 0.3000 - val_loss: 1.3855
Epoch 11/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.2368 - loss: 1.3991 - val_accuracy: 0.3000 - val_loss: 1.3868
Epoch 12/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.1842 - loss: 1.3859 - val_accuracy: 0.3000 - val_loss: 1.3850
Epoch 13/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.3947 - loss: 1.3773 - val_accuracy: 0.3000 - val_loss: 1.3813
Epoch 14/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.4474 - loss: 1.3582 - val_accuracy: 0.3000 - val_loss: 1.3525
Epoch 15/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.4737 - loss: 1.2245 - val_accuracy: 0.4000 - val_loss: 1.2317
```

Fig. 7.2: Training log (Epochs 1–15)

```
Epoch 16/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5263 - loss: 0.9840 - val_accuracy: 0.4000 - val_loss: 1.4233
Epoch 17/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.4737 - loss: 0.9461 - val_accuracy: 0.4000 - val_loss: 1.6738
Epoch 18/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5526 - loss: 0.8932 - val_accuracy: 0.4000 - val_loss: 1.9673
Epoch 19/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5000 - loss: 1.1469 - val_accuracy: 0.3000 - val_loss: 1.7630
Epoch 20/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.5000 - loss: 0.9894 - val_accuracy: 0.3000 - val_loss: 1.5537
Epoch 21/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.4737 - loss: 0.9388 - val_accuracy: 0.4000 - val_loss: 1.2034
Epoch 22/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.4211 - loss: 1.0456 - val_accuracy: 0.4000 - val_loss: 1.4011
Epoch 23/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.5000 - loss: 0.9278 - val_accuracy: 0.3000 - val_loss: 1.7366
Epoch 24/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.5000 - loss: 0.9405 - val_accuracy: 0.3000 - val_loss: 1.5216
Epoch 25/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.4474 - loss: 1.0546 - val_accuracy: 0.3000 - val_loss: 1.5489
Epoch 26/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.5000 - loss: 0.9589 - val_accuracy: 0.5000 - val_loss: 1.1674
Epoch 27/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.4474 - loss: 0.9113 - val_accuracy: 0.4000 - val_loss: 1.4482
Epoch 28/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.5526 - loss: 0.8884 - val_accuracy: 0.4000 - val_loss: 1.8745
Epoch 29/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.4211 - loss: 1.1342 - val_accuracy: 0.4000 - val_loss: 1.8387
Epoch 30/30
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 0.5789 - loss: 1.0686 - val_accuracy: 0.4000 - val_loss: 1.2355
```

Fig. 7.3: Training log (Epochs 16–30)

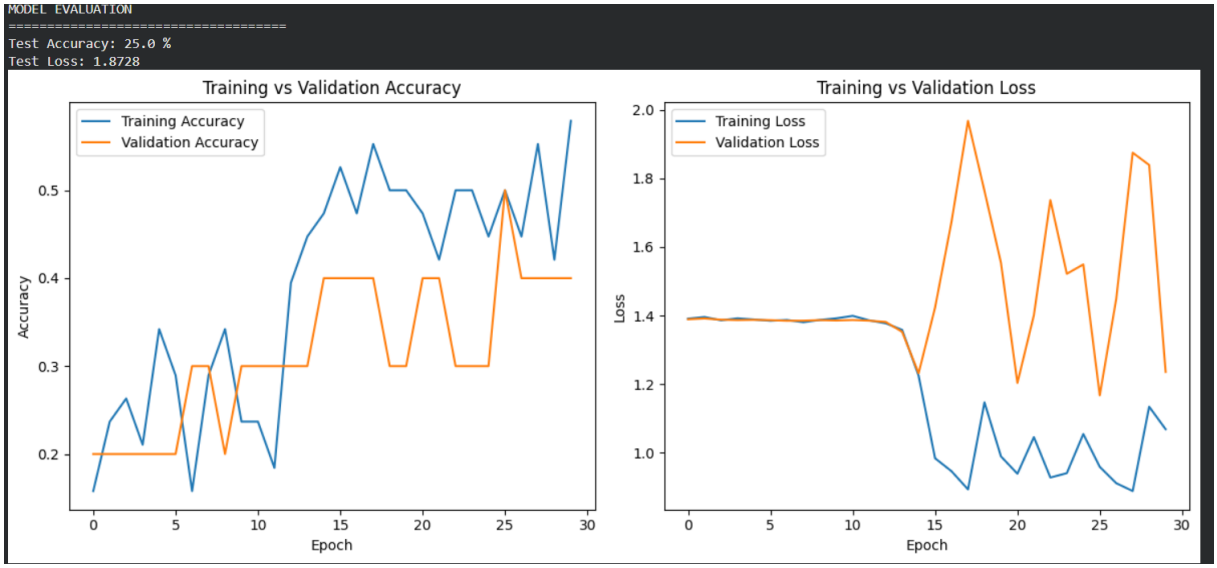


Fig. 7.4: Test accuracy, test loss and training vs validation curves

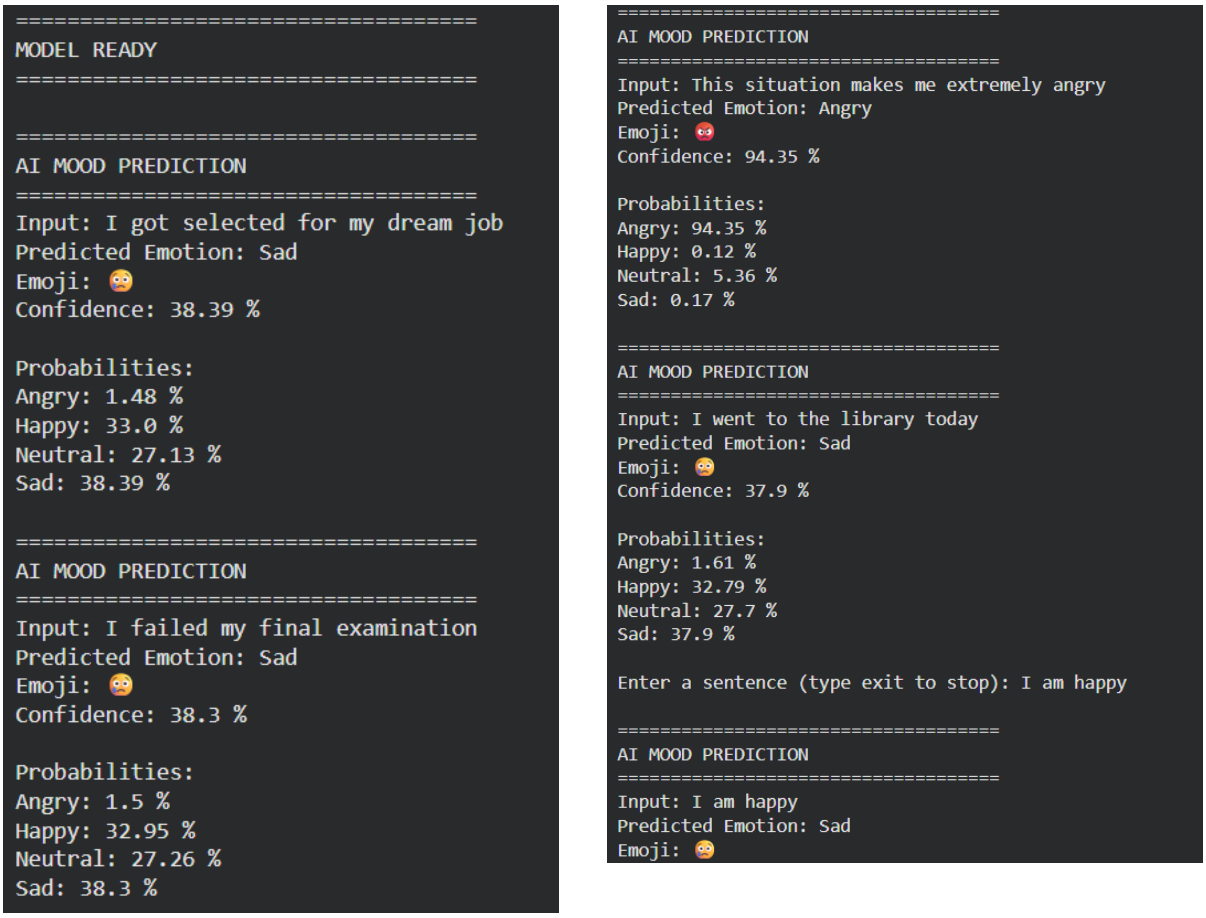


Fig. 7.5: Predictions for the sample sentences

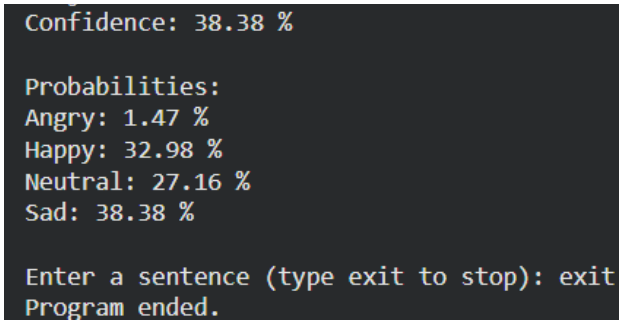


Fig. 7.6: Prediction for the sentence typed by the user and program exit

```
================================================
MODEL PERFORMANCE EVALUATION
================================================
Accuracy          : 0.2500 (25.00%)
Macro Precision   : 0.0625
Macro Recall      : 0.2500
Macro F1-Score    : 0.1000
Weighted F1-Score : 0.1000
Test Loss         : 1.8728

CLASSIFICATION REPORT
              precision    recall  f1-score   support

       Angry     0.0000    0.0000    0.0000         3
       Happy     0.0000    0.0000    0.0000         3
     Neutral     0.0000    0.0000    0.0000         3
         Sad     0.2500    1.0000    0.4000         3

    accuracy                         0.2500        12
   macro avg     0.0625    0.2500    0.1000        12
weighted avg     0.0625    0.2500    0.1000        12
```

Fig. 7.7: Performance evaluation and classification report

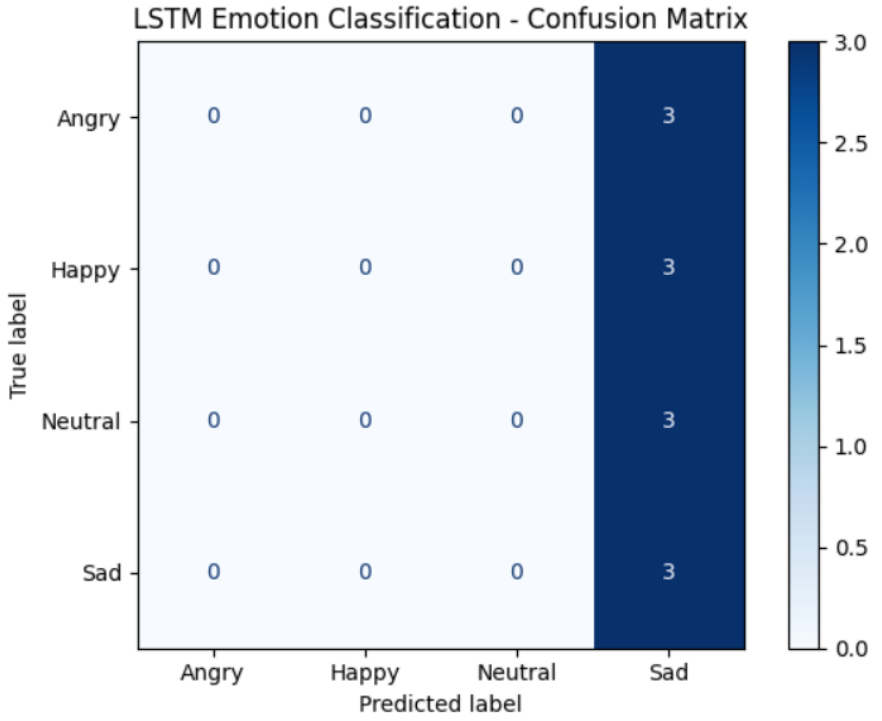


Fig. 7.8: Confusion matrix

CLASS-WISE METRICS

	Precision	Recall	F1-Score
Angry	0.00	0.0	0.0
Happy	0.00	0.0	0.0
Neutral	0.00	0.0	0.0
Sad	0.25	1.0	0.4

Fig. 7.9: Class-wise metrics

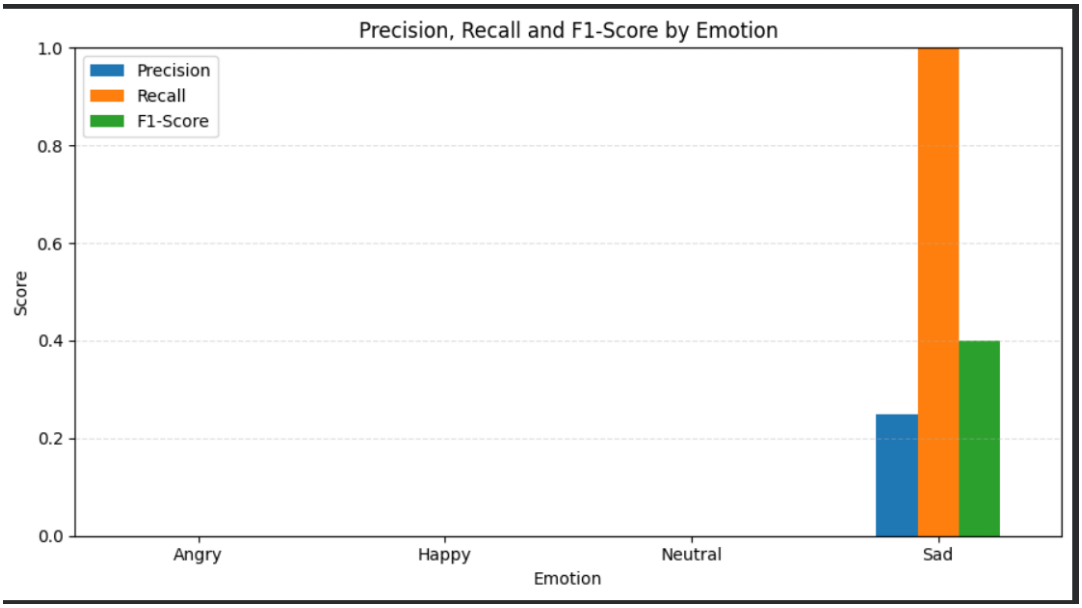


Fig. 7.10: Precision, recall and F1-score by emotion

REFERENCES

[1] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013. Available at: https://arxiv.org/abs/1301.3781

[3] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv:1412.6980, 2014. Available at: https://arxiv.org/abs/1412.6980

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014. Available at: https://jmlr.org/papers/v15/srivastava14a.html

[5] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available at: https://jmlr.org/papers/v12/pedregosa11a.html

[6] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” arXiv:1603.04467, 2016. Available at: https://arxiv.org/abs/1603.04467

[7] Keras. LSTM layer – Keras API Documentation. Available at: https://keras.io/api/layers/recurrent_layers/lstm

[8] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020. Available at: https://doi.org/10.1038/s41586-020-2649-2

[9] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[10] W. McKinney, “Data Structures for Statistical Computing in Python,” in Proceedings of the 9th Python in Science Conference, 2010, pp. 56–61.

[11] The dataset used in this project was created and collected by the authors specifically for the proposed mood prediction system.